

NES APU — Guide Complet pour Débutants

(Audio Processing Unit — L'unité de traitement audio)

Table des Matières

Introduction : Qu'est-ce que l'APU ?

Architecture Générale

Les 5 Canaux Audio

3.1 Pulse 1 & Pulse 2 (Carrés)

3.2 Triangle

3.3 Noise (Bruit)

3.4 DMC (Échantillonnage)

Les Registres de l'APU

Les Enveloppes et le Volume

Les Sweeps (Glissement de Fréquence)

Le Frame Counter (Compteur de Trame)

Le Mélangeur (Mixer)

Le Buffer Audio et la Sortie

Implémentation Pas à Pas

Tableau Récapitulatif des Registres

Checklist d'Implémentation

1. Introduction : Qu'est-ce que l'APU ?

L'**APU** (Audio Processing Unit) est le circuit audio de la NES. C'est le petit musicien qui fait tous les sons du jeu : les sauts de Mario, les tirs, la musique de fond, les explosions...

Ce que l'APU NES peut faire :

- Jouer de la musique avec 2 voix "carrées" (comme des synthétiseurs rétro)
- Jouer une voix "triangle" (pour les basses profondes)
- Faire du bruit blanc (pour les explosions, les pas, les percussions)
- Jouer des échantillons audio pré-enregistrés (voix, effets spéciaux)

Ce qu'elle NE peut PAS faire :

- Jouer du MP3 ou du WAV (c'est trop lourd pour 1983)
- Faire de la stéréo (la NES est en mono, les 5 canaux sont mélangés)
- Jouer plus de 5 sons en même temps

Analogie Simple

Imagine que l'APU est un **petit groupe de musique de 5 musiciens** :

Musicien	Instrument	Rôle dans le jeu
Pulse 1	Synthétiseur carré	Mélodie principale
Pulse 2	Synthétiseur carré	Accompagnement, effets
Triangle	Contrebasse	Basses profondes, kicks
Noise	Batterie électronique	Explosions, percussions, hiss
DMC	Lecteur d'échantillons	Voix, effets spéciaux

2. Architecture Générale

```

CPU  

(6502) █  

(écrit dans les registres)  

█  

█  

▼  

APU █ Pulse 1  

Pulse 2 Triangle Noise DMC █ $4000-4003 $4004-4007 $4008-400B  

$400C-$4010- 400F 4013  

Mixer ← Additionne les 5 (Analog) canaux en un signal  

Sortie Audio (Mono, 48kHz)
```

Points clés à retenir :

- Le CPU écrit dans des **registres** (adresses mémoire spéciales) pour contrôler l'APU
- Chaque canal a ses propres registres (4 à 5 registres par canal)
- L'APU génère un **échantillon audio** à chaque cycle d'horloge (environ 1.79 MHz)
- Les échantillons sont mélangés (additionnés) pour créer le son final
- Le résultat est un signal **mono** (pas de gauche/droite)

3. Les 5 Canaux Audio

3.1 Pulse 1 & Pulse 2 (Ondes Carrées)

Qu'est-ce qu'une onde carrée ?

Imagine un interrupteur qui s'allume et s'éteint très rapidement :

Amplitude
 ▲
 ■■■■■■ ■■■■■■ ■■■■■■
 ■ ■ ■ ■ ■ ■ ■■
 ■ ■ ■ ■ ■ ■ ■■

■■■■■ ■■■■■■ ■■■■■■ ■■■■■■→ Temps
 ■■■■ ■■■■ ■■■■

C'est un signal qui alterne entre "haut" (1) et "bas" (0). Plus l'alternation est rapide, plus le son est **aigu**. Plus elle est lente, plus le son est **grave**.

Le "Duty Cycle" (Rapport Cyclique)

L'APU NES ne fait pas des carrés parfaits. Elle peut changer la proportion "haut/bas" :

[illegible]

Chaque "forme" donne un son légèrement différent. Le duty 2 (50%) est le plus "pur", les autres sont plus "nasillards".

Les Registres du Canal Pulse

Chaque canal pulse a **4 registres** :

Registre	Adresse	Rôle
\$4000 / \$4004	Contrôle	Volume, enveloppe, duty cycle
\$4001 / \$4005	Sweep	Glissement de fréquence (monte/descend)
\$4002 / \$4006	Timer Low	Fréquence (partie basse)
\$4003 / \$4007	Timer High	Fréquence (partie haute) + longueur

Registre \$4000 (Contrôle) — Bit par bit :

Bit : 7 6 5 4 3 2 1 0
 D D L C V
 V V V
 u u e o o o o o
 t t n n l l l l
 y y g s u u u u
 t t m m m m
 a e e e e
 Bits 6-7 (DD) :
 Duty Cycle (0-3)
 Bit 5 (L) : Length Counter Halt (arrête le compteur de longueur)
 Bit 4 (C) : Constant Volume (1 = volume fixe, 0 = envelope)
 Bits 0-3 (VVVV) : Volume (0-15) ou Envelope Period

Exemple : Si tu écris 0x4F dans \$4000 :

- Duty = 1 (25%)
- Length counter halt = 0
- Constant volume = 1 (volume fixe)
- Volume = 15 (maximum)

Registre \$4002 / \$4006 (Timer Low) :

Bits 0-7 : Les 8 bits de poids faible de la fréquence

Registre \$4003 / \$4007 (Timer High) :

Bit : 7 6 5 4 3 2 1 0
 L L L T T
 T T T
 e e e i i i i
 n n n m m m m
 g g g e e e e
 t t t r r r r
 h h
 h
 Bits 5-7 (LLL) :
 Longueur du son (lookup table)
 Bits 0-4 (TTTTT) : Les 5 bits de poids fort de la fréquence

Calcul de la fréquence :

Timer = (TimerHigh << 8) | TimerLow
 Fréquence = CPU_CLOCK / (16 * (Timer + 1))

Avec CPU_CLOCK = 1,789,773 Hz :

- Timer = 0 → Fréquence = 1,789,773 / 16 = 111,860 Hz (très aigu, inaudible)
- Timer = 1000 → Fréquence = 1,789,773 / (16 * 1001) ≈ 111.7 Hz (grave)
- Timer = 500 → Fréquence ≈ 223.5 Hz (moyen)

Plus le Timer est **grand**, plus la fréquence est **basse** (grave).

Plus le Timer est **petit**, plus la fréquence est **haute** (aigu).

3.2 Triangle

Qu'est-ce qu'une onde triangulaire ?

Amplitude ▲ ▣ ▢ ▤ ▥ ▦ ▧ ▨ ▩ → Temps

C'est un signal qui monte et descend en **ligne droite** (pas en marches comme le carré). Il donne un son plus **doux**, plus **rond**, parfait pour les basses.

Particularités du Canal Triangle

- **Pas d'enveloppe** : Le volume est toujours au maximum (ou éteint)
- **Pas de sweep** : Pas de glissement de fréquence
- **Pas de duty cycle** : La forme est toujours triangulaire
- **Volume fixe** : On ne peut pas changer le volume (c'est tout ou rien)

Les Registres du Canal Triangle

Registre	Adresse	Rôle
\$4008	Contrôle	Linear counter (contrôle de la longueur)
\$4009	—	Inutilisé
\$400A	Timer Low	Fréquence (partie basse)
\$400B	Timer High	Fréquence (partie haute) + longueur

Registre \$4008 :

Bit 7 (C) : Linear Counter Control (1 = ne s'arrête jamais) Bits 0-6 (RRRRRR) : Linear Counter Reload Value (0-127)

Le "Linear Counter" est comme un compte à rebours. Quand il atteint 0, le canal s'éteint. C'est une autre façon de contrôler la durée du son (en plus du Length Counter).

3.3 Noise (Bruit)

Qu'est-ce que le bruit ?

Le canal Noise ne joue pas de notes. Il génère du **bruit aléatoire** :



C'est parfait pour les **explosions**, les **pas**, les **percussions**, le **souffle** (hiss).

Comment le NES génère du bruit ?

Il utilise un **registre à décalage avec rétroaction** (Linear Feedback Shift Register — LFSR). C'est une astuce matérielle qui génère une séquence pseudo-aléatoire :



À chaque cycle, le registre décale tous ses bits vers la droite. Le nouveau bit (à gauche) est le **XOR** de deux bits spécifiques du registre. Cette rétroaction crée une séquence qui semble aléatoire.

Mode "normal" : XOR des bits 0 et 1 → séquence de 32767 étapes avant de se répéter

Mode "court" : XOR des bits 0 et 6 → séquence de 93 étapes → son plus "métallique"

Les Registres du Canal Noise

Registre	Adresse	Rôle
\$400C	Contrôle	Volume, enveloppe, mode
\$400D	—	Inutilisé
\$400E	Timer	Fréquence du bruit + mode (normal/court)
\$400F	Length	Longueur du son

Registre \$400E :

Bit 7 (M) : Mode (0 = normal, 1 = court)
Bits 0-3 (PPPP) : Période du timer (lookup table, pas calcul direct)

La période n'est pas un nombre brut. C'est un index dans une table prédéfinie :

Index	Période (Mode Normal)	Période (Mode Court)
0	4	4
1	8	8
2	16	16
3	32	32
4	64	64
5	96	96
6	128	128
7	160	160
8	202	202
9	254	254
10	380	380
11	508	508
12	762	762
13	1016	1016
14	2034	2034
15	4068	4068

Plus la période est **grande**, plus le bruit est **grave** (les changements sont plus lents).

Plus la période est **petite**, plus le bruit est **aigu** (les changements sont plus rapides).

3.4 DMC (Delta Modulation Channel)

Qu'est-ce que le DMC ?

Le DMC est le canal le plus spécial. Il ne génère pas de formes d'onde. Il lit des **échantillons audio pré-enregistrés** depuis la mémoire du jeu (la cartouche).

C'est comme un **lecteur audio numérique très primitif** :

- Échantillons de 1 bit (oui, juste 0 ou 1 !)
- Taux d'échantillonnage très bas (environ 4-33 kHz)
- Qualité très basse mais reconnaissable

Comment fonctionne la Delta Modulation ?

Au lieu de stocker la valeur absolue de chaque échantillon, on stocke la **différence** (delta) par rapport au précédent :

```
Valeur actuelle : 50<br/>Échantillon DMC : +1 → Nouvelle valeur : 51<br/>Échantillon DMC : +1 →  
Nouvelle valeur : 52<br/>Échantillon DMC : -1 → Nouvelle valeur : 51<br/>Échantillon DMC : +1 →  
Nouvelle valeur : 52
```

Chaque bit du flux DMC dit : "monte d'un cran" (1) ou "descend d'un cran" (0). Le résultat est une forme d'onde en "escalier" qui approxime le son original.

Les Registres du Canal DMC

Registre	Adresse	Rôle
\$4010	Contrôle	IRQ enable, loop, fréquence
\$4011	Direct Load	Charge directe du compteur (volume)
\$4012	Sample Address	Adresse de début de l'échantillon
\$4013	Sample Length	Longueur de l'échantillon

Registre \$4010 :

```
Bit 7 (I) : IRQ Enable (1 = génère une interruption quand l'échantillon finit)<br/>Bit 6 (L) :  
Loop (1 = boucle l'échantillon)<br/>Bits 0-3 (FFFF) : Fréquence d'échantillonnage (lookup table)
```

Fréquences DMC :

Index	Fréquence (NTSC)	Fréquence (PAL)
-------	------------------	-----------------

0	4181.6 Hz	4177.4 Hz
1	4709.9 Hz	4696.4 Hz
2	5264.0 Hz	5261.4 Hz
3	5593.0 Hz	5579.2 Hz
4	6257.9 Hz	6023.9 Hz
5	7046.3 Hz	7044.9 Hz
6	7919.4 Hz	7917.2 Hz
7	8363.4 Hz	8397.0 Hz
8	9419.9 Hz	9340.8 Hz
9	11248.6 Hz	11233.8 Hz
10	12648.0 Hz	12648.5 Hz
11	13982.6 Hz	13982.3 Hz
12	16884.8 Hz	16884.6 Hz
13	21306.8 Hz	21315.5 Hz
14	24858.0 Hz	24858.0 Hz
15	33143.8 Hz	33143.8 Hz

Registre \$4012 (Sample Address) :

Adresse réelle = $0xC000 + (\text{valeur} * 64)$

Les échantillons DMC sont toujours dans la plage \$C000-\$FFFF.

Registre \$4013 (Sample Length) :

Longueur réelle = $(\text{valeur} * 16) + 1$

Le Buffer DMC

Le DMC a un petit buffer interne de **8 octets**. Quand il est vide, le DMC demande au CPU de charger un octet depuis la mémoire. Cela peut **ralentir le CPU** ! C'est pourquoi certains jeux ont des ralentissements quand le DMC joue.

4. Les Registres de l'APU — Vue d'Ensemble

Adresse	Registre	Canal
\$4000 | Pulse 1 Control | Pulse 1
\$4001 | Pulse 1 Sweep | Pulse 1
\$4002 | Pulse 1 Timer Low | Pulse 1
\$4003 | Pulse 1 Timer High | Pulse 1
\$4004 | Pulse 2 Control | Pulse 2
\$4005 | Pulse 2 Sweep | Pulse 2
\$4006 | Pulse 2 Timer Low | Pulse 2
\$4007 | Pulse 2 Timer High | Pulse 2
\$4008 | Triangle Control | Triangle
\$4009 | – (inutilisé) | –
\$400A | Triangle Timer Low | Triangle
\$400B | Triangle Timer High | Triangle
\$400C | Noise Control | Noise
\$400D | – (inutilisé) | –
\$400E | Noise Timer/Mode | Noise
\$400F | Noise Length | Noise
\$4010 | DMC Control | DMC
\$4011 | DMC Direct Load | DMC
\$4012 | DMC Sample Address | DMC
\$4013 | DMC Sample Length | DMC
\$4015 | Status / Control | Tous
\$4017 | Frame Counter | Tous (timing)

Registre \$4015 (Status / Channel Control)

En écriture (activation des canaux) :

Bit : 7 6 5 4 3 2 1 0
D N T 2 1
M o r P P C i i u u s
a l l e n s s g e e l
2 1
Pulse 1<Bit 3 (2) : Active Pulse 2<Bit 2 (T) : Active Triangle<Bit 1 (N) : Active Noise<Bit 0 (D) : Active DMC

En lecture (état des canaux) :

Les mêmes bits indiquent si le canal est actuellement en train de jouer.

Registre \$4017 (Frame Counter)

Bit 7 (M) : Mode (0 = 4-step, 1 = 5-step)
Bit 6 (I) : IRQ Inhibit (1 = désactive l'interruption)

Le Frame Counter contrôle le **rythme** de l'APU. Il génère des "ticks" réguliers pour :

- Mettre à jour les enveloppes (volume)
- Mettre à jour les sweeps (glissement)
- Décrémenter les length counters
- Décrémenter le linear counter (triangle)

Mode 4-step (standard) :

Step 1 (3722.5 cycles) : Clock Envelope + Triangle Linear
Step 2 (3722.5 cycles) : Clock Envelope + Triangle Linear + Length + Sweep
Step 3 (3722.5 cycles) : Clock Envelope + Triangle Linear
Step 4 (3722.5 cycles) : Clock Envelope + Triangle Linear + Length + Sweep + SET FRAME IRQ

Mode 5-step (alternatif) :

Step 1 : Clock Envelope + Triangle Linear + Length + Sweep
Step 2 : Clock Envelope + Triangle Linear
Step 3 : Clock Envelope + Triangle Linear + Length + Sweep
Step 4 :

(rien)
Step 5 : Clock Envelope + Triangle Linear

Registre \$4000, bit 4 (C) :
C = 1 → Volume constant (bits 0-3 = volume direct)
C = 0 → Enveloppe active (bits 0-3 = période de l'enveloppe)

Si la nouvelle fréquence dépasse 2047 (ou descend en dessous de 0), le canal est **muté** (silencieux).
C'est pour éviter les ultrasons.

7. Le Frame Counter (Compteur de Trame)

Rôle

Le Frame Counter est le **métronomie** de l'APU. Il ne génère pas de son directement, mais il envoie des signaux réguliers aux autres composants pour qu'ils se mettent à jour.

Fréquence

Sur NTSC, le Frame Counter "tique" toutes les **7457.5 cycles CPU** (environ 240 Hz).

Les Deux Modes

Mode 4-step (bit 7 de \$4017 = 0) :

```
Cycles : 0 7457 14913 22371 29829  
■ ■ ■ ■  
▼ ▼ ▼ ▼  
Action : Start Env+Lin  
Env+Lin Env+Lin Env+Lin  
+Len+Swp +Len+Swp +FrameIRQ  
Durée totale : ~29830 cycles  
CPU (~16.6 ms = 60 Hz)
```

Mode 5-step (bit 7 de \$4017 = 1) :

```
Cycles : 0 7457 14913 22371 29829 37281  
■ ■ ■ ■ ■  
▼ ▼ ▼ ▼ ▼  
Action : Start  
Env+Lin Env+Lin Env+Lin (rien) Env+Lin  
+Len+Swp +Len+Swp  
Durée totale : ~37282  
cycles CPU (~20.8 ms = 48 Hz)
```

Env+Lin = Clock Envelope + Clock Linear Counter

Len+Swp = Clock Length Counter + Clock Sweep

FrameIRQ = Déclenche une interruption CPU (si activé)

Implémentation

```
struct FrameCounter {  
    cycle_count: u32, // Compteur de cycles CPU  
    step: u8, // Étape  
    actuelle (0-3 ou 0-4)  
    mode: bool, // false = 4-step, true = 5-step  
    irq_inhibit: bool,  
    // true = pas d'interruption  
}  
  
impl FrameCounter {  
    fn step(&mut self, cycles:  
u32) {  
        self.cycle_count += cycles;  
        // Vérifier si on a atteint le prochain  
        seuil  
        let threshold = match self.step {  
            0 => 7457,  
            1 => 14913,  
            2 => 22371,  
            3 => if self.mode { 29829 } else { 29829 },  
            4 => 37281, // Seulement en mode  
            5-step  
            _ => unreachable!(),  
        };  
        if self.cycle_count >= threshold {  
            self.clock_components();  
            self.step += 1;  
            if self.mode && self.step > 4 {  
                self.step = 0;  
                self.cycle_count -= 37281;  
            } else if !self.mode && self.step > 3 {  
                self.step = 0;  
                self.cycle_count -= 29829;  
            }  
            fn  
            clock_components(&mut self) {  
                // À chaque étape, clock l'enveloppe et le linear counter  
                self.clock_envelopes();  
                self.clock_linear_counter();  
                // Aux étapes 1 et 3 (mode  
                4) ou 0 et 2 (mode 5),  
                // clock aussi les length counters et sweeps  
                let  
                clock_length_and_sweep = match (self.mode, self.step) {  
                    (false, 1) | (false, 3) => true, //  
                    Mode 4, étapes 1 et 3  
                    (true, 0) | (true, 2) => true, // Mode 5, étapes 0 et 2  
                    _ =>  
                    false,  
                };  
                if clock_length_and_sweep {  
                    self.clock_length_counters();  
                }  
            }  
        }  
    }  
}
```



```
self.clock_sweeps();<br/> }<br/><br/> // En mode 4, étape 3 : déclencher l'IRQ<br/> if
!self.mode && self.step == 3 && !self.iqr_inhibit {<br/> self.trigger_frame_iqr();<br/> }<br/>
}<br/>}
```

8. Le Mélangeur (Mixer)

Principe

Le mélangeur additionne les sorties des 5 canaux. Mais ce n'est pas une simple addition ! Sur le vrai hardware NES, le mélange est **analogique** et **non-linéaire**.

Formule de Mélange

Il existe plusieurs approximations. La plus courante est celle de Blargg :

```
// Canaux pulse (linéaire)
pulse_out = 95.52 / (8128.0 / (pulse1 + pulse2) + 100.0)
// Canaux tnd (triangle, noise, dmc - non-linéaire)
tnd_out = 163.67 / (24329.0 / (3 * triangle + 2 * noise + dmc) + 100.0)
// Sortie finale (0.0 à 1.0)
final_output = pulse_out + tnd_out
```

Pourquoi cette formule bizarre ?

- Le hardware NES utilise des résistances qui créent une courbe non-linéaire
- Les canaux pulse sont sur un circuit séparé des canaux tnd
- Cette formule approxime fidèlement le comportement analogique

Implémentation Simple (linéaire)

Pour commencer, tu peux utiliser un mélangeur linéaire plus simple :

```
fn mix(pulse1: f32, pulse2: f32, triangle: f32, noise: f32, dmc: f32) -> f32 {
  // Normaliser chaque canal (0.0 à 1.0)
  let pulse1_norm = pulse1 / 15.0;
  let pulse2_norm = pulse2 / 15.0;
  let triangle_norm = triangle / 15.0;
  let noise_norm = noise / 15.0;
  let dmc_norm = dmc / 127.0;
  // Mélange linéaire simple
  let mixed = (pulse1_norm + pulse2_norm + triangle_norm + noise_norm + dmc_norm) / 5.0;
  // Limiter entre 0.0 et 1.0
  mixed.min(1.0).max(0.0)
}
```

Puis passe à la formule non-linéaire quand tout fonctionne.

9. Le Buffer Audio et la Sortie

Fréquence d'Échantillonnage

L'APU NES génère un échantillon à chaque cycle CPU (1.79 MHz). Mais ta carte son veut du **48 kHz** (ou 44.1 kHz). Il faut donc **sous-échantillonner**.

Sous-Échantillonnage (Downsampling)

Cycles CPU par échantillon audio :
 $1,789,773 \text{ Hz} / 48,000 \text{ Hz} \approx 37.3 \text{ cycles}$
Donc tu génères un échantillon toutes les ~37 cycles CPU.

Implémentation avec Buffer Circulaire

```
const SAMPLE_RATE: f32 = 48000.0;  
const CPU_CLOCK: f32 = 1789773.0; // NTSC  
const CYCLES_PER_SAMPLE: f32 = CPU_CLOCK / SAMPLE_RATE; // ~37.3  
struct Apu {  
    // ... tous les canaux ...  
    sample_counter: f32, // Compteur fractionnaire  
    audio_buffer: Vec<f32>, // Buffer circulaire  
}  
impl Apu {  
    fn step(&mut self, cpu_cycles: u32) {  
        // Mettre à jour l'APU pour chaque cycle CPU  
        for _ in 0..cpu_cycles {  
            self.clock_all_channels();  
            self.frame_counter.step(1);  
            // Accumuler pour le sous-échantillonnage  
            self.sample_counter += 1.0;  
            if self.sample_counter >= CYCLES_PER_SAMPLE {  
                self.sample_counter -= CYCLES_PER_SAMPLE;  
                // Générer un échantillon audio  
                let sample = self.generate_sample();  
                self.audio_buffer.push(sample);  
            }  
        }  
    }  
    fn generate_sample(&self) -> f32 {  
        let pulse1 = self.pulse1.output();  
        let pulse2 = self.pulse2.output();  
        let triangle = self.triangle.output();  
        let noise = self.noise.output();  
        let dmc = self.dmc.output();  
        // Formule de mélange non-linéaire  
        let pulse_out = 95.52 / (8128.0 / (pulse1 + pulse2) + 100.0);  
        let tnd_out = 163.67 / (24329.0 / (3.0 * triangle + 2.0 * noise) + 100.0);  
        pulse_out + tnd_out  
    }  
}
```

Lecture du Buffer par la Carte Son

```
// Avec rodio (crate Rust pour l'audio)  
use rodio::{OutputStream, Sink, Source};  
let (_stream, stream_handle) = OutputStream::try_default().unwrap();  
let sink = Sink::try_new(&stream_handle).unwrap();  
// Créer une source audio personnalisée  
let source = ApuAudioSource::new(apu_buffer);  
sink.append(source);
```

10. Implémentation Pas à Pas

Étape 1 : Créer la structure de base

```
pub struct Apu {  
  pulse1: PulseChannel,  
  pulse2: PulseChannel,  
  triangle: TriangleChannel,  
  noise: NoiseChannel,  
  dmc: DmcChannel,  
  frame_counter: FrameCounter,  
  status: u8, // $4015  
  frame_counter_reg: u8, // $4017  
  sample_counter: f32,  
  audio_buffer: Vec<f32>,  
}
```

Étape 2 : Implémenter chaque canal

Canal Pulse (simplifié) :

```
struct PulseChannel {  
  // Registres  
  control: u8, // $4000 / $4004  
  sweep: u8, // $4001 / $4005  
  timer_low: u8, // $4002 / $4006  
  timer_high: u8, // $4003 / $4007  
  // État interne  
  timer: u16, // Compteur de fréquence (11 bits)  
  timer_period: u16, // Valeur rechargée dans timer  
  sequence_step: u8, // Position dans la séquence duty (0-7)  
  length_counter: u8, // Compteur de longueur  
  envelope: Envelope,  
  sweep_unit: SweepUnit,  
  enabled: bool,  
}  
  
impl PulseChannel {  
  fn clock_timer(&mut self) {  
    if self.timer == 0 {  
      self.timer = self.timer_period;  
    } else {  
      self.timer -= 1;  
    }  
  }  
  
  fn output(&self) -> u8 {  
    // Si le canal est désactivé, mute  
    if !self.enabled || self.length_counter == 0 {  
      return 0;  
    }  
  
    // Si le timer est < 8, mute (ultrasons)  
    if self.timer_period < 8 {  
      return 0;  
    }  
  
    // Si le sweep mute, silence  
    if self.sweep_unit.mute() {  
      return 0;  
    }  
  
    // Lookup table du duty cycle  
    const DUTY_TABLE: [[u8; 8]; 4] = [  
      [0, 1, 0, 0, 0, 0, 0, 0], // 12.5%  
      [0, 1, 1, 0, 0, 0, 0, 0], // 25%  
      [0, 1, 1, 1, 1, 0, 0, 0], // 50%  
      [1, 0, 0, 1, 1, 1, 1, 1], // 75% (inversé)  
    ];  
  
    let duty = (self.control >> 6) & 3;  
    let sequence_value = DUTY_TABLE[duty as usize][self.sequence_step as usize];  
  
    if sequence_value == 0 {  
      return 0;  
    }  
  
    // Retourner le volume (constant ou envelope)  
    if self.control & 0x10 != 0 {  
      // Volume constant  
      self.control & 0x0F  
    } else {  
      // Envelope  
      self.envelope.volume()  
    }  
  }  
}
```

Étape 3 : Connecter au Bus CPU

```
impl Apu {  
  pub fn read(&mut self, address: u16) -> u8 {  
    match address {  
      0x4015 => {  
        // Status register (lecture)  
        let mut status = 0;  
        if self.pulse1.length_counter > 0 { status |= 0x01; }  
        if self.pulse2.length_counter > 0 { status |= 0x02; }  
        if self.triangle.length_counter > 0 { status |= 0x04; }  
        if self.noise.length_counter > 0 { status |= 0x08; }  
        if self.dmc.bytes_remaining > 0 { status |= 0x10; }  
        status  
      }  
      _ => 0,  
    }  
  }  
  
  pub fn write(&mut self, address: u16, value: u8) {  
    match address {  
      0x4000..=0x4003 => self.pulse1.write(address - 0x4000, value),  
      0x4004..=0x4007 => self.pulse2.write(address - 0x4004, value),  
      0x4008..=0x400B => self.triangle.write(address - 0x4008, value),  
      0x400C..=0x400F => self.noise.write(address - 0x400C, value),  
      0x4010..=0x4013 => self.dmc.write(address - 0x4010, value),  
      0x4015 => {  
        // Status / Channel control  
        self.pulse1.enabled = value & 0x01 != 0;  
        self.pulse2.enabled = value & 0x02 != 0;  
        self.triangle.enabled = value & 0x04 != 0;  
        self.noise.enabled = value & 0x08 != 0;  
        self.dmc.set_enabled(value & 0x10 != 0);  
      }  
      0x4017 => {  
        self.frame_counter_reg = value;  
        self.frame_counter.set_mode(value & 0x80 != 0);  
        self.frame_counter.set_irq_inhibit(value & 0x40 != 0);  
      }  
      _ => {},  
    }  
  }  
}
```

Étape 4 : Intégrer dans l'Emulateur

```
// Dans ton Bus ou Emulator<br/>struct Emulator {<br/> cpu: Cpu,<br/> ppu: Ppu,<br/> apu:<br/> Apu,<br/> bus: Bus,<br/> // ...<br/>}<br/><br/>impl Emulator {<br/> fn run_frame(&mut self)<br/> {<br/> // Le PPU fait 3 cycles pour 1 cycle CPU<br/> // L'APU fait 1 cycle par cycle CPU<br/> while !self.ppu.frame_complete() {<br/> let cpu_cycles = self.cpu.step();<br/><br/> for _ in<br/> 0..cpu_cycles * 3 {<br/> self.ppu.step();<br/> }<br/><br/> self.apu.step(cpu_cycles);<br/><br/> }<br/> }<br/>}
```

11. Tableau Récapitulatif des Registres

Adresse	Nom	Bits	Description
\$4000	Pulse 1 Ctrl	DDLC VVVV	Duty, LenHalt, ConstVol, Vol/Env
\$4001	Pulse 1 Sweep	EPPP NSSS	Enable, Period, Negate, Shift
\$4002	Pulse 1 TimerL	TTTT TTTT	Timer Low (8 bits)
\$4003	Pulse 1 TimerH	LLLL LTTT	Length, Timer High (5 bits)
\$4004	Pulse 2 Ctrl	DDLC VVVV	Même que \$4000
\$4005	Pulse 2 Sweep	EPPP NSSS	Même que \$4001
\$4006	Pulse 2 TimerL	TTTT TTTT	Même que \$4002
\$4007	Pulse 2 TimerH	LLLL LTTT	Même que \$4003
\$4008	Triangle Ctrl	CRRR RRRR	LinearCtrl, LinearCount
\$4009	—	—	Inutilisé
\$400A	Triangle TimerL	TTTT TTTT	Timer Low
\$400B	Triangle TimerH	LLLL LTTT	Length, Timer High
\$400C	Noise Ctrl	—LC VVVV	(pas de duty), LenHalt, ConstVol, Vol/Env
\$400D	—	—	Inutilisé
\$400E	Noise Timer	M—P PPPP	Mode, Period (lookup)
\$400F	Noise Length	LLLL L—	Length (pas de timer high)
\$4010	DMC Ctrl	IL— FFFF	IRQ, Loop, Frequency
\$4011	DMC Load	DDDD DDDD	Direct load (volume)
\$4012	DMC Address	AAAA AAAA	Sample address (\$C000 + A*64)
\$4013	DMC Length	LLLL LLLL	Sample length (L*16 + 1)
\$4015	Status	—DNT 21—	Write=enable, Read=status
\$4017	Frame Counter	MI— ———	Mode, IRQ Inhibit

12. Checklist d'Implémentation

Phase 1 : Structure de Base

- ☐ Créer le module `apu.rs`
- ☐ Définir la structure `Apu` avec les 5 canaux
- ☐ Connecter les registres \$4000-\$4017 au bus CPU
- ☐ Implémenter `read()` et `write()` pour chaque registre

Phase 2 : Canaux Simples

- ☐ Implémenter le Timer pour Pulse 1 (génération d'onde carrée)
- ☐ Implémenter le Duty Cycle
- ☐ Implémenter le Volume Constant
- ☐ Tester avec un ROM simple (ex: test ROM APU)

Phase 3 : Envelopes et Length Counters

- ☐ Implémenter l'Envelope
- ☐ Implémenter le Length Counter (lookup table)
- ☐ Connecter au Frame Counter
- ☐ Tester le decay du volume

Phase 4 : Sweeps

- ☐ Implémenter le Sweep Unit
- ☐ Tester le glissement de fréquence
- ☐ Vérifier le mute condition

Phase 5 : Autres Canaux

- ☐ Implémenter Pulse 2 (copier Pulse 1)
- ☐ Implémenter Triangle (pas d'enveloppe, linear counter)
- ☐ Implémenter Noise (LFSR)
- ☐ Implémenter DMC (optionnel, plus complexe)

Phase 6 : Frame Counter

- ☐ Implémenter le Frame Counter 4-step
- ☐ Implémenter le Frame Counter 5-step
- ☐ Connecter les clocks aux canaux

Phase 7 : Mélangeur et Sortie Audio

- ☐ Implémenter le mélangeur (formule linéaire d'abord)
- ☐ Implémenter le sous-échantillonnage
- ☐ Connecter à une bibliothèque audio (rodio, cpal, etc.)
- ☐ Tester avec un vrai jeu

Phase 8 : Polish

- ☐ Passer au mélangeur non-linéaire
- ☐ Optimiser les performances
- ☐ Ajouter le DMC si pas déjà fait
- ☐ Tester avec plusieurs jeux

Ressources Utiles

Documentation Technique

- **NesDev Wiki** (wiki.nesdev.com) — La bible de l'émulation NES
- **APU Reference** (nesdev.com/wiki/APU) — Détails techniques complets
- **Blargg's APU Tests** — ROMs de test pour valider ton APU

Test ROMs

- **apu_test.nes** — Tests basiques de l'APU
- **apu_mixer.nes** — Tests du mélangeur
- **apu_reset.nes** — Tests du reset
- **apu_dmc.nes** — Tests du canal DMC

Bibliothèques Audio Rust

- **rodio** — Simple, haut niveau
- **cpal** — Bas niveau, cross-platform
- **miniaudio** — Performant, bindings C

Conclusion

L'APU de la NES est un circuit audio **simple mais élégant**. Il ne fait que 5 canaux, mais avec les enveloppes, les sweeps, et le mélangeur non-linéaire, il produit une grande variété de sons caractéristiques.

Conseils pour l'implémentation :

Commence simple : Pulse 1 avec volume constant, pas d'enveloppe

Teste à chaque étape : Utilise les ROMs de test de Blargg

Compare avec un émulateur de référence : Mesen, FCEUX

N'oublie pas le timing : Le Frame Counter est crucial

Le DMC peut attendre : C'est le canal le plus complexe, fais-le en dernier

Bonne chance avec ton émulateur ! ■■